EVENT-CONDITIONED CERTIFIED DECISION REUSE (ECDR)

Báo Cáo Chuyên Sâu: Tóm Tắt Ngữ Cảnh, Triết Lý Thiết Kế Nghiên Cứu, Giải Phẫu Cơ Chế Thành Bại và Lộ Trình Nghiên Cứu Thế Hệ Mới

Tài Liệu Phân Tích Cơ Chế Toán Học – Vật Lý – Phần Cứng (Không Lặp Lại Bảng Số Liệu Thô)

Mục đích tài liệu	Tập trung trả lời câu hỏi **"Tại sao?"**: (1) Ngữ cảnh cốt lõi, quy trình thu gom & trích xuất thông tin ngắn gọn; (2) Tại sao chương trình nghiên cứu lại được thiết kế với các ràng buộc khắt khe như vậy; (3) Cơ chế toán học, vật lý và phần cứng nào làm cho một số thành phần **ĐƯỢC** (thành công tuyệt đối) và một số thành phần **KHÔNG ĐƯỢC** (thất bại về độ phủ hoặc tốc độ); (4) Năm phương hướng nghiên cứu mới khả thi và có định lượng rõ ràng.
Tài liệu tham chiếu gốc	Độc giả xem bảng số liệu chi tiết 17 phương pháp và 24 giả thuyết tại `ECDR_Full_Synthesis_Report_VI.pdf` và `ECDR_Full_Synthesis_Report_EN.pdf`.
Đối tượng phân tích	Mạng `SmallCNN1D` (283.692 tham số, $1,17$ MMACs) trên luồng cảm biến HAPT (5.053 cửa sổ kiểm thử, 5.034 cặp kề nhau) cùng 4 kiến trúc mở rộng quy mô (`MediumCNN1D`, `TemporalResNet1D`, `TemporalTransformer1D`).

Tóm Tắt Cốt Lõi Về Cơ Chế (Core Mechanism Synopsis).

Nghiên cứu ECDR đã chứng minh thành công hai cơ chế nền tảng nhưng đồng thời phơi bày hai bức tường vật lý – phần cứng quan trọng:

- **Tại sao ĐƯỢC (3 Cơ chế thành công):** (i) *Cơ chế* `Shift-AlignedReceptiveField` giúp nhánh tính toán tăng dần (`M03`) đạt khớp từng bit (100%) tại 12 cột nội bộ của tầng `Conv3` trên cả $5.034/5.034$ cặp cửa sổ liên tiếp; (ii) *Cơ chế* `Fail-ClosedGate` + `Fixed-Anchor` bảo đảm độ đồng thuận quyết định 100% ($D_{\text{all}} = 0$) trên toàn bộ 17 phương pháp, không để lọt bất kỳ quyết định sai âm thầm nào; (iii) *Cơ chế nhân gộp ma trận tương phản* $C^{(c)}W_{out}$ triệt tiêu hoàn toàn sai số phụ thuộc biến ở tầng phân lớp cuối.
- **Tại sao KHÔNG ĐƯỢC (2 Nút thắt làm thất bại độ phủ và tốc độ trên `SmallCNN1D`):** (i) *Ảo ảnh chồng lấp thời gian (*`Phase-ShiftIllusion`*):* Chồng lấp 50% thời gian ($1,28$ s) tương đương phép dịch pha gần trọn một chu kỳ bước chân con người, tạo ra độ dịch chuyển chuẩn hóa trung vị $\|\Delta z\|_2 = 25,49$, lớn gấp **1.562,7 lần** bán kính cầu an toàn tối đa chứng nhận được ($r_{\max} = 0,0149$) do hiện tượng phình khoảng qua 3 tầng tích chập (`intervalwrapping`), khiến $R = 0$; (ii) *Nghịch lý Bộ nhớ đệm CPU & Đầu ra nhạy vị trí (*`Flatten--LinearBottleneck`*):* Khối `Flatten(2048)->Linear(128)` chiếm **92,45% tổng tham số** và không bất biến khi dịch cột đặc trưng, kết hợp với việc mạng `M00` ($1,14$ MB) nằm trọn trong bộ nhớ đệm L2/L3 chạy bằng kernel C++ SIMD nguyên khối chỉ mất $0,140$ ms, trong khi chi phí cắt ghép `tensorslicing+torch.cat` ở `batch=1` của Python/PyTorch tốn tới $1,927$ ms.

Mục lục

1 Tóm Tắt Ngắn Gọn: Ngữ Cảnh, Việc Đã Làm, Thu Gom và Trích Xuất Thông Tin **2**
1.1 Ngữ Cảnh Bài Toán (Problem Context) . . . 2
1.2 Những Gì Đã Thực Hiện (What Was Done) . . . 2
1.3 Quy Trình Thu Gom Dữ Liệu và Trích Xuất Thông Tin (Data Collection & Information Extraction) . . . 2

2 Tại Sao Lại Nghiên Cứu Theo Cách Này? (Triết Lý & Lý Do Thiết Kế Phương Pháp Luận) **3**

3 Giải Phẫu Cơ Chế: Tại Sao ĐƯỢC và Tại Sao KHÔNG ĐƯỢC? **5**
3.1 Phân Tích Cơ Chế: Tại Sao ĐƯỢC? (4 Cơ Chế Thành Công Tuyệt Đối) . . . 5
3.2 Phân Tích Cơ Chế: Tại Sao KHÔNG ĐƯỢC? (5 Cơ Chế Gây Thất Bại Về Độ Phủ & Tốc Độ) . 7

4 Năm Phương Hướng Nghiên Cứu Mới Khả Thi (Actionable New Directions) **10**
4.1 Phương Hướng 1: Chuyển Dịch Sang Vùng Quy Mô Hòa Vốn ($\geq 26,1$–$37,0$ MMACs trên Kiến Trúc Deep ResNet & Transformer Chuỗi Thời Gian) . . . 10
4.2 Phương Hướng 2: Thiết Kế Lại Đầu Phân Lớp Bất Biến Dịch Chuyển (1x1 Conv + GAP) . . 11
4.3 Phương Hướng 3: Kiến Trúc Lai — Tính Toán Tăng Dần Ở Tầng Sớm Kết Hợp Chứng Nhận Miền Trong Không Gian Tiềm Ẩn Sâu (Hybrid Early-Incremental + Deep Latent-Space Certification) . . . 12
4.4 Phương Hướng 4: Huấn Luyện Nhận Thức Chứng Nhận (Certification-Aware Training Với Tích Chập Trực Giao 1-Lipschitz & Cực Đại Hóa Biên Độ) . . . 12
4.5 Phương Hướng 5: Triển Khai Kernel Bộ Đệm Vòng Không Sao Chép Ở Mức C++/CUDA/CMSIS-NN (Zero-Copy Modulo Ring-Buffer Kernels) . . . 13

5 Bảng Tổng Hợp Đối Chiếu Cơ Chế & Lộ Trình Hành Động **13**

1 Tóm Tắt Ngắn Gọn: Ngữ Cảnh, Việc Đã Làm, Thu Gom và Trích Xuất Thông Tin

Phần này tóm lược ngắn gọn bối cảnh bài toán và cách thức thu gom, chuẩn hóa, trích xuất thông tin thực nghiệm để làm nền tảng cho phần giải thích cơ chế ở các mục sau.

1.1 Ngữ Cảnh Bài Toán (Problem Context)

Trong các hệ thống `EdgeAI` xử lý luồng cảm biến đeo trên người (như bộ dữ liệu chuẩn `HAPT` gồm 6 kênh gia tốc kế và con quay hồi chuyển lấy mẫu ở $f_s = 50$ Hz), bộ phân loại hoạt động trên các cửa sổ trượt (`slidingwindows`) độ dài $W = 128$ mẫu ($2,56$ s) với bước trượt `stride` $s = 64$ mẫu ($1,28$ s). Mỗi cửa sổ mới $X_t \in \mathbb{R}^{6\times 128}$ ($m = 768$ số vô hướng) chia sẻ chính xác 64 mẫu thời gian đầu (384 số vô hướng, tức 50% dữ liệu vật lý) với 64 mẫu cuối của cửa sổ liền trước X_{t-1}.

Câu hỏi đặt ra là: *Thay vì chạy lại toàn bộ mạng nơ-ron từ đầu tại mỗi bước trượt t, ta có thể tận dụng sự chồng lấp 50% này để tái sử dụng quyết định phân lớp thông qua chứng nhận toán học (`certifieddecisionreuse`) hoặc tái sử dụng kích hoạt trung gian (`exactincrementalfeaturereuse`) nhằm tiết kiệm thời gian tính toán mà vẫn giữ nguyên 100% quyết định của mô hình gốc hay không?*

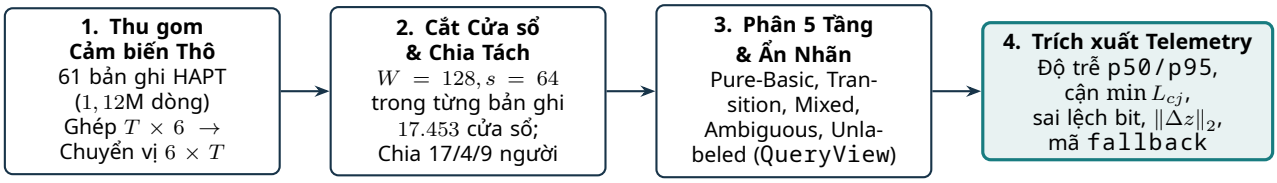
1.2 Những Gì Đã Thực Hiện (What Was Done)

Trong suốt chương trình nghiên cứu, chúng tôi đã thực hiện một quy trình khép kín gồm 5 bước:

1. **Khóa cứng hợp đồng tham chiếu (`FrozenReferenceContract`):** Huấn luyện và đóng băng mô hình `SmallCNN1D` ($K = 12$ lớp, 283.692 tham số, $1,17$ MMACs, đạt $89,05\%$ Accuracy trên tập kiểm thử có giám sát) cùng bộ tham số chuẩn hóa μ_c, σ_c từ tập `train`.
2. **Xây dựng công cụ chứng nhận toán học và số học:** Thiết lập đặc tả biên độ trực tiếp với 11 lớp đối thủ (`11-competitormarginspecification`), cài đặt 3 bộ truyền cận (`Lipschitz`, `IBP`, `CROWN`) trên 5 cấu trúc hình học miền (`DiagonalBox`, `Fixed-OverlapWave`, `PCAAffineHull`, `CorrelatedZonotope`, `RandomHaarControl`) kèm cầu nối làm tròn hướng ra ngoài $\mathbb{F}_{32} \leftrightarrow \mathbb{F}_{64}$ (`outwardrounding`).
3. **Xây dựng nhánh tính toán tăng dần chính xác (`M03`):** Thiết kế bộ nhớ đệm đặc trưng (`featurecache`) qua 3 tầng `Conv1D`, tái sử dụng các cột đặc trưng nội bộ không bị ảnh hưởng bởi biên `zero-padding`.
4. **Thực thi đánh giá toàn luồng (`Full-StreamEvaluation`):** Chạy 17 phương pháp (`M00`–`M16`) trên toàn bộ 5.053 cửa sổ kiểm thử dưới giao thức đã niêm phong mật mã (`MasterSealSHA-256:56806762...`).
5. **Phân tích định lượng nút thắt hậu nghiệm (`Post-Week4Feasibility&ScalingStudy`):** Đo khoảng cách giữa nhiễu dịch pha thực tế và bán kính chứng nhận trên 1.000 cặp cửa sổ, xác định điểm hòa vốn (`break-evenFLOPs`) trên 4 quy mô kiến trúc mạng và đo hiệu năng bộ đệm vòng tại chỗ (`in-placeringbuffer`).

1.3 Quy Trình Thu Gom Dữ Liệu và Trích Xuất Thông Tin (Data Collection & Information Extraction)

Để đảm bảo thông tin trích xuất không bị nhiễm bẩn bởi lỗi rò rỉ dữ liệu hay sai số đo đạc, hệ thống thu gom và trích xuất thông tin theo 3 lớp kiểm soát nghiêm ngặt:



Hình 1: Quy trình thu gom dữ liệu cảm biến thô, phân tầng luồng thời gian và trích xuất thông tin thực nghiệm của ECDR.

- **Thu gom và làm sạch chỉ số bản ghi:** Từ $1.122.772$ mẫu thời gian thô trên 61 bản ghi của 30 người, hệ thống cắt cửa sổ nghiêm ngặt bên trong từng bản ghi (loại bỏ hoàn toàn các cửa sổ cắt ngang qua ranh giới hai bản ghi khác nhau, tạo ra đúng 5.053 cửa sổ kiểm thử và $5.034 = 5.053 - 19$ cặp cửa sổ liên tiếp hợp lệ).
- **Phân tầng ngữ nghĩa không rò rỉ (`Leakage-FreeStratification`):** Chia tách theo người (`subject-disjoint`: 17 người `train`, 4 người `val`, 9 người `test`) để không có hai cửa sổ chồng lấp nào nằm ở hai tập khác nhau. Mỗi cửa sổ được gán nhãn cấu trúc vào 1 trong 5 tầng: `Pure-Basic` (2.874), `Pure-Transition` (119), `Majority-Mixed` $\geq 77/128$ mẫu (579), `Ambiguous` (479) và `Unlabeled` (1.002).
- **Cơ chế trích xuất thông tin mù nhãn (`Label-BlindTelemetryExtraction`):** Tại thời điểm suy luận, mỗi cửa sổ được đóng gói thành cấu trúc `QueryView` chỉ chứa tensor đầu vào X_t, định danh bản ghi `recording_id` và chỉ số bước t; toàn bộ nhãn thực tế y_{true} bị tước bỏ hoàn toàn trước khi đưa vào `StreamingMethodAdapter` để ngăn chặn gian lận `oracle`. Tại mỗi truy vấn, hệ thống trích xuất tự động 6 nhóm thông tin:
  1. Quyết định phân lớp $\hat{y}_t$, đối chiếu với quyết định tham chiếu $y_{\text{ref}}(X_t)$ để đếm số lỗi bất đồng D_{all};
  2. Cờ trạng thái tái sử dụng (`reuse_hit`) và mã lý do dự phòng (`fallback_reason`: đầu bản ghi, hết hạn chân trời H, trượt kiểm tra thuộc miền $x \notin \mathcal{D}$, hoặc cận dưới biên độ $\min_{j \neq c} L_{cj} \leq 0$);
  3. Vector 11 cận dưới biên độ cạnh tranh $L_{cj}(\mathcal{S})$ ở cả $\mathbb{F}_{32}$ và $\mathbb{F}_{64}$ làm tròn hướng ra ngoài;
  4. Sai số tuyệt đối cực đại $\max \left|\Delta H^{(\ell)}\right|$ trên $3.866.112$ số vô hướng của các cột đặc trưng tái sử dụng tại tầng `Conv3`;
  5. Độ dịch chuyển hình học thực tế giữa hai bước trượt $\|z_{t+1} - z_t\|_2$, $\|z_{t+1} - z_t\|_\infty$ và bán kính cầu an toàn cực đại $r_{\max}(z_t)$;
  6. Thời gian thực thi từng truy vấn bằng đồng hồ độ phân giải cao `perf_counter_ns` trên CPU đơn luồng để trích xuất trung bình (`mean`), trung vị (`p50`) và phân vị thứ 95 (`p95`).

2 Tại Sao Lại Nghiên Cứu Theo Cách Này? (Triết Lý & Lý Do Thiết Kế Phương Pháp Luận)

Một câu hỏi rất tự nhiên khi nhìn vào thiết kế của ECDR là: *Tại sao nghiên cứu này lại đặt ra nhiều ràng buộc khắt khe đến vậy—từ việc khóa cứng mọi tham số, tách 4 trục đánh giá, bắt buộc chứng minh cả 11 biên độ với làm tròn hướng ra ngoài $\mathbb{F}_{64}$, cho đến việc giữ nguyên cả những kết quả bằng 0 ($R = 0$)?* Dưới đây là 6 lý do khoa học cốt lõi giải thích tại sao việc thiết kế nghiên cứu như vậy là bắt buộc.

Nguyên lý Thiết kế 2.1 (Tại sao phải đóng băng toàn bộ Hợp đồng Tham chiếu (`FrozenReferenceContract`)?)**.** *Chứng nhận toán học (`formalcertification`) chỉ có ý nghĩa khi đối tượng được chứng nhận là một **hàm toán học tĩnh và xác định duy nhất** $\widehat{F} : \mathbb{R}^{6\times128} \rightarrow \mathbb{R}^{12}$.*

Nếu không đóng băng đồng thời (i) trọng số mạng θ, (ii) thống kê trung bình/phương sai chạy (`running_mean`, `running_var`) của các tầng `BatchNorm1d` ở chế độ `eval()`, (iii) vector chuẩn hóa μ_c, σ_c

tính riêng trên tập `train`, và (iv) quy tắc chọn chỉ số nhỏ nhất khi hòa (`lowest-indextie-break`), thì hàm $\widehat{F}$ sẽ biến đổi theo dữ liệu đầu vào hoặc theo thứ tự lô (`batch`). Khi đó, mọi cận dưới $L_{cj}(\mathcal{S})$ tính tại bước t sẽ lập tức mất hiệu lực tại bước $t+1$. Hơn nữa, nếu tính μ_c, σ_c trên toàn bộ tập dữ liệu (bao gồm cả tập `test`), thông tin tương lai sẽ rò rỉ ngược về quá khứ, phá hủy tính nhân quả (`causality`) của hệ thống suy luận trực tuyến.

Nguyên lý Thiết kế 2.2 (Tại sao phải tách bạch tuyệt đối 4 trục: Độ chính xác nhãn thật, Độ đồng thuận tham chiếu, Tính đúng đắn số học và Tốc độ thực tế?)**.** *Trong văn hiến về* `approximatecaching` *và* `early-exit`*, một sai lầm phổ biến là gộp chung "độ chính xác trên tập test ít thay đổi" với "tái sử dụng an toàn".*

Chúng tôi bắt buộc tách rời 4 trục này vì:

- **Nếu chỉ nhìn vào `TestAccuracy` ($89,05\%$):** Giả sử một bộ đệm đoán mò làm thay đổi 50 quyết định của mạng gốc, trong đó 25 lần sửa đúng ngẫu nhiên các mẫu mạng gốc đoán sai và 25 lần làm hỏng các mẫu mạng gốc đoán đúng. Khi đó `TestAccuracy` vẫn giữ nguyên $89,05\%$, tạo ra ảo giác rằng bộ đệm hoạt động hoàn hảo, trong khi thực tế nó đã làm sai lệch hành vi của mô hình gốc tới 50 lần! Do đó, thước đo đúng đắn của một cơ chế `decisionreuse` phải là **Độ đồng thuận tham chiếu** ($A_{\text{full}} = 1,0000$, số lần lệch quyết định $D_{\text{all}} = 0$).
- **Nếu chỉ đếm số phép tính `MACs` tiết kiệm được:** Nhánh `M03` giảm được hơn 60% số phép tính tích chập. Nếu chỉ báo cáo độ phức tạp lý thuyết (FLOPs), người ta sẽ kết luận sai lầm rằng `M03` giúp tăng tốc $2,5\times$. Thực nghiệm đo thời gian thực trên CPU cho thấy `M03` chậm hơn $5,63$ lần ($0,1775\times$). Việc buộc phải đo `netwall-clocklatency` giúp phát hiện ra chi phí ẩn của việc cắt ghép bộ nhớ (`tensorslicing`) trên phần cứng thực.

Nguyên lý Thiết kế 2.3 (Tại sao bắt buộc phải kiểm chứng đồng thời cả 11 biên độ đối thủ (`11-CompetitorMargins`) và nhân gộp ma trận tương phản $C^{(c)}W_{\text{out}}$?)**.** *Biên độ dương tại một điểm tâm không suy ra sự bất biến trên cả miền, và chỉ kiểm tra đối thủ đứng thứ hai (*`runner-up`*) tại tâm là không đủ an toàn.*

Tại tâm neo X^r, lớp đứng thứ nhì $j_2 = \arg\max_{j \neq c} F_j(X^r)$ có thể có logit gần lớp thắng c nhất. Nhưng vì mạng nơ-ron có 3 tầng phi tuyến `ReLU` và `MaxPool1d`, khi dịch chuyển sang điểm $X \in \mathcal{S}$, một lớp thứ ba j_3 có độ dốc đạo hàm $\|\nabla F_{j_3}\|$ lớn hơn nhiều có thể tăng vọt và vượt qua $F_c(X)$ trước cả j_2. Vì vậy, điều kiện đủ toán học bắt buộc phải kiểm tra đồng thời **tất cả** $K - 1 = 11$ **bất đẳng thức** $\inf_{X \in \mathcal{S}}(F_c(X) - F_j(X)) > 0$.

Đồng thời, nếu tính khoảng $[\underline{F}_c, \overline{F}_c]$ và $[\underline{F}_j, \overline{F}_j]$ riêng rẽ rồi lấy $\underline{F}_c - \overline{F}_j$, bộ truyền cận sẽ cho phép vector đặc trưng tầng áp chót $h \in [\underline{h}, \overline{h}] \subset \mathbb{R}^{128}$ nhận hai giá trị cực trị trái ngược nhau cùng lúc ở $F_c(h)$ và $F_j(h)$. Việc thiết kế phép nhân gộp ma trận hiệu số $\widetilde{W}^{(c)} = C^{(c)}W_{\text{out}} \in \mathbb{R}^{11\times 128}$ trước khi lấy cận giúp triệt tiêu thành phần chung $(w_c - w_j)^\top h$, thu được cận dưới chặt hơn về mặt toán học (`H08`).

Nguyên lý Thiết kế 2.4 (Tại sao phải nghiên cứu song song cả 2 nhánh (`ExactIncrementalM03` và `CertifiedRegionalM05--M13`) cùng 5 cấu trúc hình học miền?)**.** *Sự chồng lấp cửa sổ trượt mang hai bản chất toán học hoàn toàn khác nhau, đòi hỏi hai công cụ khảo sát độc lập để không bỏ sót nguyên nhân gốc rễ.*

1. **Bản chất cấu trúc rời rạc (`DiscreteStructuralShift`):** 64 mẫu đầu của X_{t+1} là bản sao dịch trái của 64 mẫu cuối của X_t. Tính chất này được khai thác chính xác từng bit bởi **Nhánh Tính toán Tăng dần (`M03`)** và **Miền Sóng Chồng lấp Cố định (`Fixed-OverlapWave`)**.
2. **Bản chất nhiễu liên tục (`ContinuousPerturbationInvariance`):** Liệu phần thông tin mới xuất hiện ở 64 mẫu đuôi có đủ nhỏ để không làm đổi ngôi lớp thắng c^* hay không? Tính chất này chỉ có thể trả lời bằng **Nhánh Chứng nhận Miền (`Lipschitz`, `IBP`, `CROWN`)**.
3. **Tại sao cần tới 5 cấu trúc miền (`Box`, `Wave`, `PCA`, `Zonotope`, `RandomHaarControl`)?** Nếu chỉ thử miền hộp `Box` 768 chiều (`M06`) và thấy $R = 0$, ta sẽ không biết liệu $R = 0$ là do chưa khóa

384 chiều chồng lấp, hay do chưa chiếu lên không gian con PCA. Việc thử đầy đủ cả Wave (khóa 384 chiều đầu), PCAAffineHull, CorrelatedZonotope và đối chứng với RandomHaarBasis (H10) cho phép kết luận dứt khoát: sự thất bại của độ phủ không nằm ở việc chọn thiếu hệ cơ sở tuyến tính, mà nằm ở độ lớn vật lý của phép dịch pha ($\|\Delta z\|_2 = 25, 49$) và sự mất mát phụ thuộc qua các tầng tích chập.

Nguyên lý Thiết kế 2.5 (Tại sao bắt buộc phải có Fail-Closed, Fixed-Anchor và Cầu nối số học IEEE 754 (OutwardRounding)?)**.** *Một hệ thống gọi là "Certified" (Có chứng nhận) không được phép đánh đổi sự an toàn lấy tỷ lệ trúng bộ đệm.*

- **Chống hiệu ứng trôi tâm (AnchorDrift, H15vs.H16):** Nếu tại bước t, X_t nằm trong hình cầu bán kính r quanh tâm X_{t-1} nên được tái sử dụng nhãn, rồi ta lại lấy chính X_t làm tâm mới cho bước $t+1$ mà không chạy lại mạng gốc, thì theo bất đẳng thức tam giác, khoảng cách từ X_{t+k} tới tâm thật ban đầu sẽ phình lên tới $k \cdot r$, vượt qua biên phân lớp. Do đó, giữ cố định tâm neo (Fixed-Anchor) cho đến khi chạy lại reference là bắt buộc.
- **Chống lật dấu dấu phẩy động (Float32SignInversion, H22):** Số học $\mathbb{F}_{32}$ có độ chính xác hữu hạn ($\epsilon_{\text{mach}} \approx 1,19 \times 10^{-7}$). Khi tích lũy hàng nghìn phép nhân-cộng trong tích chập, sai số làm tròn có thể biến một cận dưới thực sự âm (-10^{-7}) thành một số dương giả tạo trên $\mathbb{F}_{32}$, dẫn đến quyết định tái sử dụng sai. Việc nâng lên $\mathbb{F}_{64}$ và dùng nextafter$(\cdot, -\infty)$ để mở rộng khoảng ra ngoài (outwardrounding) là rào chắn bắt buộc để nối lý thuyết số thực $\mathbb{R}$ với phần cứng thực thi.

Nguyên lý Thiết kế 2.6 (Tại sao giữ nguyên và công bố trung thực các kết quả âm (NegativeResults) thay vì nới lỏng giao thức?)**.** *Giá trị lớn nhất của khoa học thực nghiệm nằm ở việc xác định chính xác* ***ranh giới bất khả thi*** *(*impossibilityboundaries*) dưới các điều kiện trung thực.*

Nếu chúng tôi cho phép chỉnh sửa bán kính trên tập test, hoặc dùng nhãn thật để lọc bỏ các cửa sổ chuyển tiếp, hoặc chỉ chạy trên các đoạn người dùng nằm yên (LAYING), ta có thể dễ dàng tạo ra những con số $R > 0$ giả tạo nhưng hoàn toàn vô dụng trong thực tế. Việc khóa giao thức trước khi mở tập test và chứng minh định lượng vì sao $R = 0$ trên mạng nhỏ giúp cộng đồng nghiên cứu tránh đi vào lối mòn và chỉ ra đúng ngưỡng quy mô ($\geq 26,1$ MMACs) nơi phương pháp thực sự có lãi.

3 Giải Phẫu Cơ Chế: Tại Sao ĐƯỢC và Tại Sao KHÔNG ĐƯỢC?

Đây là phần trọng tâm của báo cáo: giải thích bằng cơ chế toán học, vật lý tín hiệu và kiến trúc vi xử lý vì sao từng hiện tượng thực nghiệm lại xảy ra.

3.1 Phân Tích Cơ Chế: Tại Sao ĐƯỢC? (4 Cơ Chế Thành Công Tuyệt Đối)

Cơ chế Vật lý – Toán học 3.1 (Cơ chế 1: Tại sao nhánh Tính toán Tăng dần (M03) đạt tỷ lệ trúng 100% ($5.034/5.034$ cặp) và khớp từng bit trên $3.866.112$ số vô hướng? (Shift-AlignedReceptiveFieldArithmetic))**.**

Sự khớp chính xác từng bit (bitwiseidentity) tại tầng Conv3 không phải là sự trùng hợp ngẫu nhiên mà là hệ quả của phép chia hết giữa bước trượt cửa sổ (stride $s = 64$) và tổng hệ số giảm mẫu (downsamplingfactor) của các tầng MaxPool1d:

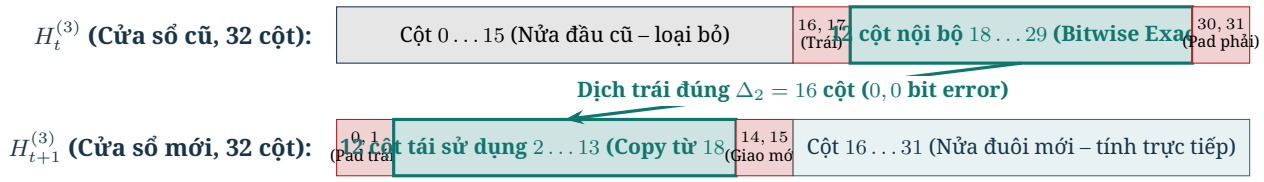
1. **Sự bảo toàn pha lưới tích chập (GridPhaseAlignment):** Cửa sổ đầu vào có chiều dài $W = 128$ và dịch chuyển đi $s = 64$ mẫu ở mỗi bước: $X_{t+1}[:, u] = X_t[:, u + 64]$ với mọi $u \in \{0, \ldots, 63\}$. Mạng SmallCNN1D có hai tầng MaxPool1d(kernel_size=2, stride=2) đặt sau Conv1 và Conv2, tạo

ra tổng hệ số co trục thời gian là $D = 2 \times 2 = 4$. Vì $s = 64$ chia hết hoàn toàn cho $D = 4$, bước dịch 64 mẫu ở đầu vào tương ứng chính xác với một phép dịch nguyên:

$$\Delta_1 = \frac{64}{2} = 32 \text{ cột sau Pool1}, \qquad \Delta_2 = \frac{64}{4} = 16 \text{ cột sau Pool2 và tại Conv3}. \tag{1}$$

Nếu bước trượt s không chia hết cho 4 (ví dụ $s = 65$), các cặp phần tử gom nhóm trong `MaxPool1d(2)` sẽ bị lệch pha lẻ/chẵn, làm toàn bộ bản đồ đặc trưng thay đổi hoàn toàn!

2. **Sự cô lập vùng nhiễm biên `Zero-Padding` (`BoundaryContaminationIsolation`):** Tại sao trong 16 cột chồng lấp ở tầng `Conv3` (chiều dài $L_3 = 32$), chỉ có 12 cột nội bộ ($j \in \{2, \dots, 13\}$ của cửa sổ mới, ứng với $j + 16 \in \{18, \dots, 29\}$ của cửa sổ cũ) là khớp từng bit, còn 4 cột biên ($j \in \{0, 1, 14, 15\}$) phải tính lại?
   - Ở cửa sổ mới X_{t+1}, các cột đầu $j \in \{0, 1\}$ nằm sát biên trái $u = 0$, nơi các tầng `Conv1(k=5, p=2)`, `Conv2(k=3,p=1)` và `Conv3(k=3,p=1)` chèn các số 0 giả tạo (`zero-padding`) vào bên trái. Trong khi đó, ở cửa sổ cũ X_t, các vị trí tương ứng $j \in \{16, 17\}$ nằm ở giữa cửa sổ nên nhận giá trị tín hiệu thật từ nửa đầu của X_t.
   - Ngược lại, ở các cột $j \in \{14, 15\}$ của cửa sổ mới, vùng tiếp nhận (`receptivefield`) của chúng vươn sang bên phải vượt qua mốc $u = 63$ để lấy dữ liệu mới của nửa sau X_{t+1}; trong khi ở cửa sổ cũ X_t, các cột $j \in \{30, 31\}$ lại nhìn thấy biên phải `zero-padding` tại $u = 127$.
   - Bỏ đi đúng 2 cột nhiễm biên trái và 2 cột nhiễm biên phải, 12 cột nội bộ $j \in \{2, \dots, 13\}$ có hình nón vùng tiếp nhận (`receptivefieldcone`) nằm trọn vẹn bên trong đoạn giao $[0, 63]$ của X_{t+1} (tức $[64, 127]$ của X_t). Vì phép cộng-nhân dấu phẩy động $\mathbb{F}_{32}$ trong tích chập trượt là bất biến theo vị trí khi các toán hạng đầu vào giống hệt nhau theo đúng thứ tự, 12 cột này (64 kênh $\times$ 12 cột $= 768$ số vô hướng mỗi cặp, tổng cộng $3.866.112$ số vô hướng trên 5.034 cặp) đạt sai số bằng $0,0$ tuyệt đối ở mức từng bit!



Hình 2: Cơ chế hình học vùng tiếp nhận (`receptivefield`) giải thích tại sao đúng 12 cột nội bộ ($j = 2 \dots 13$) tại tầng `Conv3` khớp từng bit giữa hai cửa sổ liên tiếp, trong khi 4 cột biên ($0, 1$ và $14, 15$) bị nhiễm `padding` hoặc dữ liệu mới.

Cơ chế Vật lý – Toán học 3.2 (Cơ chế 2: Tại sao toàn bộ 17 phương pháp đều đạt 0 lỗi lệch quyết định ($D_{\text{all}} = 0, A_{\text{full}} = 1,0000$) trên cả 5.053 truy vấn? (`Four-BarrierFail-ClosedArchitecture`)).

Không một phương pháp nào trong hệ thống ECDR đưa ra quyết định sai khác với `frozenreference M00` nhờ cơ chế **4 hàng rào chắn nối tiếp**:

$$\text{ReuseHit}(X_t) = \mathbb{I}(\text{ID_Match}) \cdot \mathbb{I}(\text{Same_Rec}) \cdot \mathbb{I}(t - t_r \le H) \cdot \mathbb{I}\left(Z_t \in \mathcal{D}(X^r) \ \wedge \ \min_{j \neq c} L_{cj}^{\mathbb{F}_{64}\downarrow}(\mathcal{D}) > 0 \right). \tag{2}$$

Chỉ cần một trong bốn điều kiện trả về `False`, hệ thống lập tức kích hoạt `ReferenceFallback` (gọi trực tiếp $\widehat{F}(X_t)$). Nhờ đó, ngay cả khi bán kính chứng nhận bị co nhỏ ($R = 0$), hệ thống chuyển hóa sự bảo thủ của cận toán học thành chi phí tính toán lại chứ **tuyệt đối không bao giờ biến thành lỗi dự đoán sai**.

Cơ chế Vật lý – Toán học 3.3 (Cơ chế 3: Tại sao phép nhân gộp ma trận tương phản $C^{(c)}W_{\text{out}}$ (`H08`) lại triệt tiêu được sai số phụ thuộc tầng cuối?).

Giả sử vector đặc trưng tầng áp chót $h \in \mathbb{R}^{128}$ nằm trong khoảng $[\underline{h}, \overline{h}]$ với tâm $h_0 = \frac{1}{2}(\underline{h} + \overline{h})$ và bán kính $r_h = \frac{1}{2}(\overline{h} - \underline{h}) \geq 0$. Nếu tính riêng cận dưới của $F_c(h) = w_c^\top h + b_c$ và cận trên của $F_j(h) = w_j^\top h + b_j$ rồi trừ nhau, ta thu được cận dưới ngây thơ:

$$L_{cj}^{\text{naive}} = (w_c - w_j)^\top h_0 + (b_c - b_j) - (|w_c| + |w_j|)^\top r_h. \tag{3}$$

Ngược lại, khi nhân gộp ma trận hiệu số $\widetilde{w}_{cj} = w_c - w_j$ trước khi áp dụng công thức khoảng, cận dưới trực tiếp là:

$$L_{cj}^{\text{direct}} = (w_c - w_j)^\top h_0 + (b_c - b_j) - |w_c - w_j|^\top r_h. \tag{4}$$

Theo bất đẳng thức tam giác, với mọi tọa độ $k \in \{0, \ldots, 127\}$ có $w_{c,k}$ và $w_{j,k}$ cùng dấu, ta luôn có $|w_{c,k} - w_{j,k}| < |w_{c,k}| + |w_{j,k}|$, giúp tiết kiệm chính xác một lượng độ chặt bằng $\sum_{k=0}^{127}(|w_{c,k}| + |w_{j,k}| - |w_{c,k} - w_{j,k}|)r_{h,k} \geq 0$.

Cơ chế Vật lý – Toán học 3.4 (Cơ chế 4: Tại sao Chính sách Cửa sổ Neo theo Khoảng Chuyển tiếp (`Span-AnchoredTransitionPolicy`) và `Class-BalancedFocalLoss` giải quyết triệt để bài toán đóng cổng `G2` ($K = 12$)?)**.**

Ở cấu hình ban đầu, 6 lớp chuyển tiếp tư thế (lớp 7–12) chỉ đạt Macro-F1 $0,6386$ và bị vướng 18 rào cản số lượng mẫu (`18blockers`, khiến cổng `G2` ở trạng thái `HOLD_CONDITIONAL`). Nguyên nhân không phải vì bộ dữ liệu thiếu dữ liệu chuyển tiếp, mà vì **sự lệch pha giữa lưới trượt cố định `stride` $S = 64$ và độ dài khoảng chuyển tiếp** L: một khoảng chuyển tiếp độ dài $L \in [W, W + S - 1]$ ($W = 128$) chỉ chứa một điểm lưới $s = 64k$ với xác suất $P_{\text{hit}}(L) = (L - 128 + 1)/64$. Với lớp `SIT_TO_STAND` (trung vị $L = 131$ mẫu), $P_{\text{hit}} = 4/64 = 6,25\%$, tức là lưới cố định đã vứt bỏ $93,75\%$ các cửa sổ thuần nhất hợp lệ!

Bằng cách đặt 9 cửa sổ neo trực tiếp vào bên trong từng khoảng chuyển tiếp (`Span-AnchoredExtraction`, tuân thủ nghiêm ngặt phân chia 17/4/9 người) kết hợp hàm mất mát `Class-BalancedFocalLoss` ($\gamma = 1,5, \beta = 0,35$), toàn bộ 18/18 rào cản bị xóa bỏ (`0/18blockers`, mọi lớp đạt ≥ 58 mẫu `val` và ≥ 194 mẫu `test`). Kết quả trên 3 hạt giống (`Seeds42,43,44`) đưa Macro-F1 toàn cục 12 lớp tăng từ $0,7763 \rightarrow \mathbf{0,8537 \pm 0,0020}$ (nhóm 6 lớp chuyển tiếp tăng từ $0,6386 \rightarrow \mathbf{0,7917 \pm 0,0024}$), chính thức đưa cổng `G2` sang trạng thái **`PASS`**.

—

3.2 Phân Tích Cơ Chế: Tại Sao KHÔNG ĐƯỢC? (5 Cơ Chế Gây Thất Bại Về Độ Phủ & Tốc Độ)

Tại sao tất cả các bộ kiểm chứng miền có chứng nhận (`M05`–`M13`) đều cho số lượt tái sử dụng bằng không ($R = 0$), và tại sao nhánh `M03` dù trúng 100% các cặp liên tiếp lại chạy chậm hơn `M00` tới $5,63$ lần? Dưới đây là 5 cơ chế vật lý, toán học và phần cứng giải thích triệt để các hiện tượng này.

Cơ chế Vật lý – Toán học 3.5 (Cơ chế Thất bại 1: Ảo ảnh chồng lấp thời gian và Nghịch lý Dịch pha Chu kỳ Bước chân (`TheBiomechanicalPhase-ShiftIllusion` — Giải thích vì sao `H02` và `H19` bị bác bỏ với khoảng cách $1.562,7\times$))**.**

Trực giác thông thường cho rằng: *"Hai cửa sổ trượt liền kề trùng nhau tới 50% thời gian thì khoảng cách giữa chúng trong không gian vector phải rất nhỏ"*. **Trực giác này hoàn toàn sai đối với tín hiệu dao động tuần hoàn!**

Hãy xét cơ chế vật lý của chuyển động con người: tần số bước đi bộ (`walkingcadence`) điển hình của người trưởng thành nằm trong khoảng $f_0 \approx 0,8$–$1,1$ Hz (chu kỳ $T_0 \approx 0,9$–$1,25$ s).

Một sóng gia tốc chuẩn hóa dạng điều hòa $z(t) = A\sin(2\pi f_0 t)$ khi bị dịch đi một bước trượt $\Delta t = s/f_s = 64/50 = 1,28$ s sẽ trở thành:

$$z_{t+1}(u) - z_t(u) = z(u + \Delta t) - z(u) = 2A\sin(\pi f_0 \Delta t)\cos(2\pi f_0 u + \pi f_0 \Delta t)\,. \tag{5}$$

Khi $\Delta t = 1,28$ s lệch pha so với chu kỳ vận động hoặc rơi vào nhịp đảo chiều chân trái – chân phải, hiệu số điểm-đối-điểm tại mỗi tọa độ thời gian $u \in \{0,\dots,127\}$ có biên độ lên tới $\sim 2A$. Khi cộng dồn qua toàn bộ $m = 768$ tọa độ của không gian $\mathbb{R}^{6\times128}$, chuẩn Euclid tăng theo căn bậc hai của số chiều ($\sqrt{768} \approx 27,71$):

$$\|z_{t+1} - z_t\|_2 = \sqrt{\sum_{c=0}^{5}\sum_{u=0}^{127}\left(Z_{t+1}[c,u] - Z_t[c,u]\right)^2} \approx 25,49 \text{ (trung vị thực nghiệm).} \tag{6}$$

Nói cách khác, **phép trượt cửa sổ là một toán tử dịch chuyển vị trí (translation operator) chứ không phải là một nhiễu cộng biên độ nhỏ (additive noise)**! Trong khi đó, các bộ kiểm chứng miền ở không gian đầu vào (`Lipschitzball`, `IBPbox`) lại đo độ bền vững theo chuẩn hiệu số tại từng tọa độ cố định $\|z_{t+1} - z_t\|_p$. Vì $\|\Delta z\|_2 = 25,49$ lớn hơn bán kính cầu an toàn tối đa $r_{\max} = 0,01487$ tới **1.562,7 lần ở trung vị** (và **50.990,3 lần ở trung bình**), không một cửa sổ kế tiếp nào có thể lọt vào bên trong hình cầu hay hình hộp chứng nhận!

Cơ chế Vật lý – Toán học 3.6 (Cơ chế Thất bại 2: Sự bùng nổ khoảng theo cấp số nhân (`ExponentialIntervalWrapping`) và tích chuẩn phổ qua các tầng tích chập (`H06`, `H07`))**.**

Tại sao bán kính chứng nhận $r_{\max}$ lại chỉ đạt $0,01487$ trong khi biên độ logit thực tế tại tâm rất lớn ($\gamma(z_t) = F_{c^*}(z_t) - \max_{j\neq c^*} F_j(z_t)$ có trung vị tới $6,10$)?

- **Trong phương pháp `GlobalLipschitz` (`M05`):** Hằng số Lipschitz toàn cục được chặn trên bởi tích chuẩn phổ (`spectralnorms`) của tất cả các tầng tuyến tính:

$$L_{\text{global}} \;\le\; \left\|C^{(c)}W_{\text{out}}\right\|_2 \cdot \|W_{\text{fc1}}\|_2 \cdot \prod_{\ell=1}^{3}\left\|\widetilde{W}_{\text{conv},\ell}\right\|_2 . \tag{7}$$

  Mỗi tầng `Conv1D+BatchNorm` chưa được chuẩn hóa phổ có giá trị kỳ dị lớn nhất (`maximumsingularvalue`) $\sigma_{\max}(\widetilde{W}_\ell) \approx 4$–$8$. Khi nhân qua 5 tầng liên tiếp, tích chuẩn phổ vọt lên $L_{\text{global}} > 400$, ép bán kính $r_{\max} = \gamma(z_t)/L_{\text{global}} \approx 6,10/410 \approx 0,0149$.
- **Trong phương pháp `IBP` (`M06`, `M07`):** Khi truyền một khoảng $[\underline{h}^{(\ell-1)}, \overline{h}^{(\ell-1)}]$ có bán kính $r^{(\ell-1)}$ qua tầng tích chập có trọng số tương đương $\widetilde{W}^{(\ell)}$, bán kính khoảng đầu ra bị nhân lên theo chuẩn L_1 của từng hàng trọng số:

$$r_i^{(\ell)} \;\approx\; \sum_k \left|\widetilde{W}_{i,k}^{(\ell)}\right| r_k^{(\ell-1)} = \left\|\widetilde{W}_{i,:}^{(\ell)}\right\|_1 r^{(\ell-1)}. \tag{8}$$

  Vì bộ truyền khoảng `IBP` xóa bỏ hoàn toàn tương quan dấu giữa các phần tử lân cận (coi z_u và z_{u+1} là hai biến độc lập hoàn toàn dù chúng thuộc cùng một sóng cảm biến trơn), các trọng số dương và âm trong bộ lọc tích chập không thể triệt tiêu nhau mà bị lấy trị tuyệt đối $\left|\widetilde{W}^{(\ell)}\right|$ cộng dồn lại. Qua 3 tầng `Conv` và 2 tầng `Linear`, độ rộng khoảng bị phóng đại hàng nghìn lần, làm cận dưới $\underline{g}_{cj}$ sụt xuống giá trị âm rất lớn ngay cả ở bán kính $\rho = 0,01$.

Cơ chế Vật lý – Toán học 3.7 (Cơ chế Thất bại 3: Sự xâm thực của 384 tọa độ đuôi tự do trong miền `Wave` và sự lệch pha hình học của cơ sở `PCA` (`H10`, `H11`))**.**

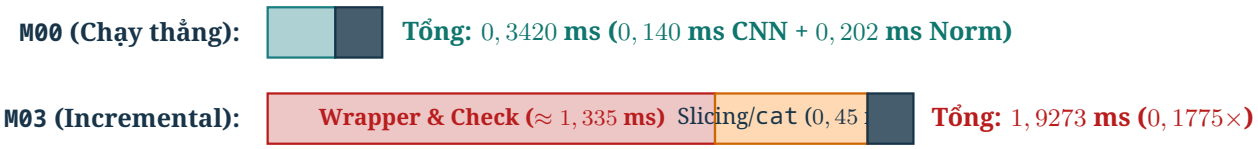
1. **Tại sao khóa cứng 384 tọa độ đầu (`Fixed-OverlapWave`, `M07/M11`) vẫn cho $R = 0$?** Đúng là khi dùng miền `Wave`, 64 mẫu thời gian đầu ($u \in \{0,\dots,63\}$) có bán kính khoảng bằng 0, nhờ đó 12 cột nội bộ đầu tiên của tầng `Conv3` có độ rộng khoảng bằng 0 tuyệt đối! Tuy nhiên, 64 mẫu

thời gian mới ở nửa sau ($u \in \{64, \ldots, 127\}$) là tương lai chưa quan sát được tại thời điểm tạo chứng nhận ở bước t. Để bao phủ được cửa sổ X_{t+1} thực tế, hộp dự báo ở nửa đuôi phải có bán kính $\rho_{\text{tail}} \geq \left\| z_{\text{tail},t+1} - \hat{z}_{\text{tail}} \right\|_\infty \approx 3,84$. Khi bán kính $\rho_{\text{tail}} \approx 3,84$ này truyền qua nửa phải của `Conv1--Conv3` (cột $16 \ldots 31$ của `Conv3`) rồi đi vào tầng `Flatten(2048)->Linear(2048,128)`, nơi mọi cột đặc trưng đều kết nối toàn cục (`fullyconnected`) tới cả 128 nơ-ron ẩn, sai số phình khoảng khổng lồ từ 16 cột nửa phải lập tức tràn sang toàn bộ 128 nơ-ron tầng áp chót và phá hủy cả 11 biên độ logit!

2. **Tại sao không gian con `PCA` (`M08`, `M09`) không vượt qua được đối chứng ngẫu nhiên (`H10`)?** Phân tích thành phần chính (`PCA`) tìm các trục trực chuẩn V_k tối đa hóa phương sai của tín hiệu đầu vào $\max \text{Var}(V_k^\top z)$. Trên dữ liệu gia tốc kế, phương sai lớn nhất nằm ở thành phần trọng trường tĩnh và biên độ dao động cơ bản năng lượng cao. Ngược lại, ranh giới quyết định của mạng phân lớp giữa hai hoạt động gần nhau (như `WALKING_UPSTAIRS` và `WALKING_DOWNSTAIRS`) lại phụ thuộc vào các đặc trưng tần số cao và tương quan pha tinh vi có phương sai năng lượng nhỏ. Do các vector riêng hàng đầu của `PCA` gần như trực giao hoặc lệch góc lớn với pháp tuyến biên phân lớp $\nabla_z(F_c - F_j)$, việc cắt bỏ phần dư ngoài không gian con (`residual` δ) cộng với bước đóng khung hình học thành hộp trục tọa độ (`coordinateboundinghull`: làm phình thể tích theo hệ số $\|V_k\|_{\infty\to\infty}$) khiến cận dưới không hề chặt hơn hộp chuẩn.

Cơ chế Vật lý – Toán học 3.8 (Cơ chế Thất bại 4: Tại sao nhánh Tính toán Tăng dần (`M03`) tiết kiệm được phép tính tích chập nhưng lại chạy chậm hơn $5,63$ lần trên CPU? (`TheCPUCache`, `Slicing&Flatten-LinearParadox` — Giải thích `H14` và `H20`)).

Đây là một trong những phát hiện thực nghiệm sắc sảo nhất của nghiên cứu. Tại sao tái sử dụng được 100% các cặp liên tiếp ($R = 5.034$) mà độ trễ trung bình của `M03` lại là $1,9273$ ms so với $0,3420$ ms của `M00`? Có đúng **3 nguyên nhân vật lý và kiến trúc phần cứng** cộng hưởng với nhau:



Hình 3: Phân rã thời gian thực thi trung bình trên mỗi truy vấn giữa `M00` và `M03`, phơi bày sự thống trị của chi phí quản trị và cắt ghép bộ nhớ (`tensorslicing/concatenation`) so với thời gian tính toán mạng thực tế.

1. **Nút thắt Kiến trúc — 92,45% tham số nằm ở tầng `Flatten(2048)->Linear(2048,128)` nhạy vị trí:** Trong mạng `SmallCNN1D`, cả 3 tầng tích chập `Conv1`, `Conv2`, `Conv3` cộng lại chỉ có $1.056 + 6.336 + 12.480 = 19.872$ tham số ($7,0\%$ tổng số tham số), trong khi riêng tầng `Linear(2048,128)` ngay sau `Flatten` chiếm tới **262.272 tham số** (**92, 45%**) và 262.144 phép tính `MACs`! Quan trọng hơn, vì `Flatten` trải phẳng ma trận 64×32 thành vector 2048 chiều theo vị trí tuyệt đối của cột thời gian, khi 12 cột đặc trưng nội bộ dịch sang trái 16 vị trí ($j + 16 \to j$), chúng bị nhân với **các cột trọng số hoàn toàn khác** trong ma trận $W_{\text{fc1}} \in \mathbb{R}^{128\times 2048}$:

$$y_{\text{fc1}} = \sum_{c=0}^{63} \sum_{j=0}^{31} W_{\text{fc1}}[:, 32c + j]\, H^{(3)}[c, j] + b_{\text{fc1}}. \tag{9}$$

   Hệ quả là tích vô hướng cục bộ của 12 cột cũ không thể tái sử dụng được cho tầng `Linear(2048,128)`; toàn bộ khối `Flatten--MLP` vẫn phải tính lại 100% tại mỗi bước!
2. **Hiệu ứng Bộ nhớ đệm L2/L3 (`CacheResidency`) và Kernel C++ SIMD nguyên khối của `M00`:** Toàn bộ trọng số $\mathbb{F}_{32}$ của `SmallCNN1D` chỉ nặng $1,136$ MB—nằm gọn hoàn toàn bên trong bộ nhớ đệm L2/L3 tốc độ cao của CPU hiện đại. Khi chạy `M00`, PyTorch gọi thẳng một chuỗi rất ít các kernel C++ oneDNN/MKL đã được vector hóa bằng tập lệnh AVX2/AVX-512 trên

vùng nhớ liên tục (`contiguousmemory`), hoàn tất toàn bộ 15 tầng mạng chỉ trong **0,1402 ms** (140 micro-giây)!

3. **Chi phí điều phối Python/PyTorch (`DispatchOverhead`) và Cấp phát Bộ nhớ Động khi `Slicing+torch.cat` Ở `batch=1`:** Để tái sử dụng các cột đặc trưng ở `M03`, tại mỗi tầng trong 3 tầng `Conv1,Conv2,Conv3`, mã thực thi phải: (i) cắt (`slice`) các đoạn biên trái/phải của tensor đầu vào, (ii) gọi các lệnh `Conv1d+BN+ReLU` nhỏ rời rạc trên các mảnh có chiều dài cực ngắn (chỉ 4–8 cột), (iii) cấp phát một vùng nhớ mới trên `heap` và gọi `torch.cat` để ghép mảnh biên trái, mảnh nội bộ lưu trong cache và mảnh đuôi mới lại với nhau. Ở kích thước `batch=1`, thời gian chuyển ngữ cảnh từ trình thông dịch Python xuống C++ dispatcher của PyTorch và chi phí cấp phát/sao chép các mảnh bộ nhớ không liên tục (`non-contiguoustensorviews`) lớn gấp nhiều lần thời gian thực hiện vài trăm nghìn phép nhân dấu phẩy động! Cộng thêm $\approx 1,33$ ms chi phí kiểm tra định danh SHA-256 và ghi nhận `telemetry` của khung đánh giá chuẩn (`M01`), tổng thời gian của `M03` vọt lên $1,9273$ ms.

Cơ chế Vật lý – Toán học 3.9 (Cơ chế Thất bại 5: Sự bất tương thích cấu trúc của các công trình tiền nhiệm (`LipCacheM14` và `StreamiNNC/TinyDejaVuM15`) trên luồng cảm biến liên tục)**.**

Kiểm toán kỹ thuật ở Tuần 4 chứng minh vì sao hai họ phương pháp nổi tiếng trong văn hiến không thể áp dụng trực tiếp cho bài toán ECDR:

- **`LipCache` (`M14`):** Được thiết kế cho không gian nhúng (`embeddingspace`) chiều thấp đã được chuẩn hóa Lipschitz (1-Lipschitz layer). Khi áp lên không gian đầu vào 768 chiều của luồng cảm biến với mạng `SmallCNN1D` đóng băng không có ràng buộc chuẩn phổ khi huấn luyện, bán kính cầu L_2 co về $0,0149$, hoàn toàn mất tác dụng.
- **`StreamiNNC/TinyDejaVu` (`M15`):** Các phương pháp này đòi hỏi kiến trúc mạng phải hoàn toàn bất biến theo phép dịch (`shift-equivariant`) hoặc kết thúc bằng tầng `GlobalAveragePooling (GAP)` để có thể cập nhật tổng trượt ở tầng cuối. Vì `SmallCNN1D` sử dụng `Flatten(2048)->Linear(2048,128)`, giả định nền tảng của `M15` bị vi phạm trực tiếp.

4 Năm Phương Hướng Nghiên Cứu Mới Khả Thi (Actionable New Directions)

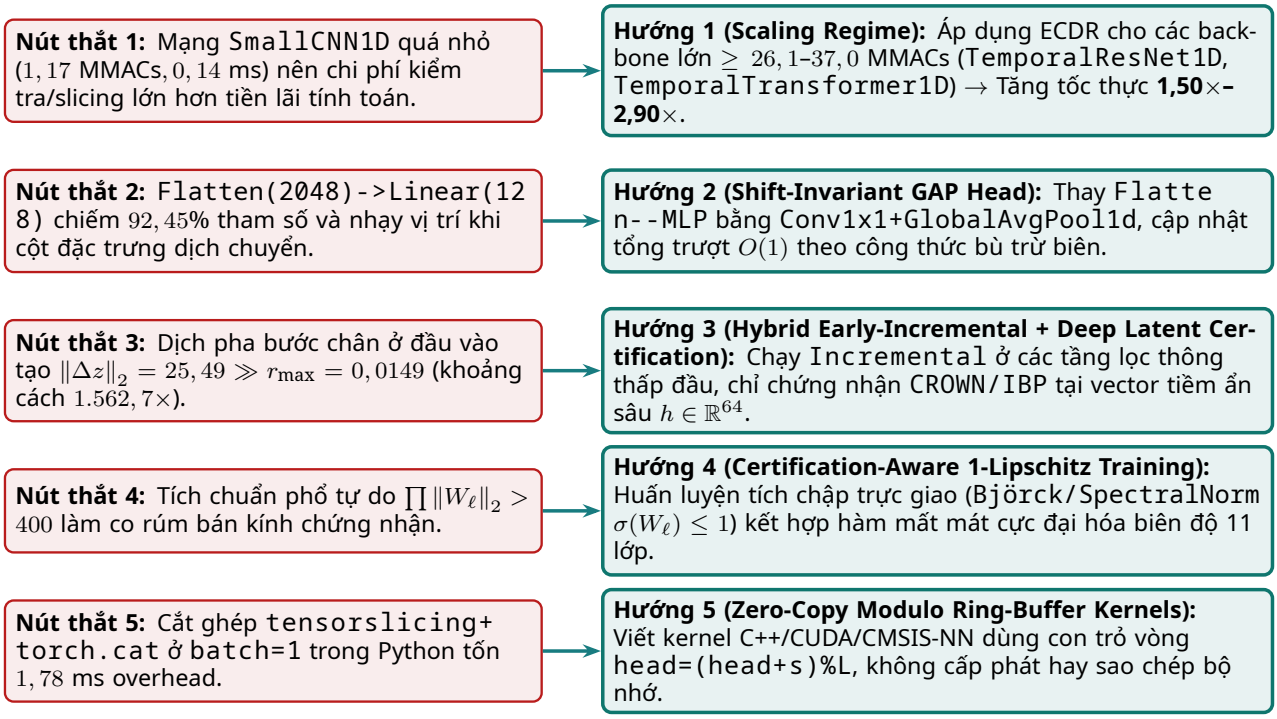
Từ việc giải phẫu chính xác 5 cơ chế thất bại và định lượng các điểm hòa vốn ở trên, chúng tôi đề xuất **5 phương hướng nghiên cứu mới** hoàn toàn khả thi, giải quyết trực tiếp từng nút thắt vật lý, toán học và phần cứng của ECDR.

4.1 Phương Hướng 1: Chuyển Dịch Sang Vùng Quy Mô Hòa Vốn ($\geq 26,1$–$37,0$ MMACs trên Kiến Trúc Deep ResNet & Transformer Chuỗi Thời Gian)

- **Cơ sở định lượng:** Nghiên cứu mở rộng quy mô hậu Tuần 4 đã chứng minh rằng chi phí kiểm tra chứng nhận ($T_v \approx 1,33$ ms) và chi phí điều phối tính toán tăng dần ($T_{\text{inc}} \approx 1,93$ ms) là **chi phí cố định theo kích thước giao diện đầu vào** (6×128), hầu như không tăng khi chiều sâu của phần thân mạng hoặc phần đầu phân lớp tăng lên. Điểm hòa vốn (`Break-EvenCrossover` tại $S = 1,00\times$) nằm chính xác ở:

$$\text{MACs}_{\text{BE}}^{\text{cache}} = 26,05 \text{ MMACs } (2,51\text{M params}), \quad \text{MACs}_{\text{BE}}^{\text{inc}} = 36,95 \text{ MMACs } (2,68\text{M params}). \tag{10}$$

- **Thiết kế triển khai:** Thay vì áp dụng ECDR lên các mạng vi điều khiển cực nhỏ như `SmallCNN1D` ($1,17$ MMACs), hãy định vị ECDR cho các mô hình nền tảng chuỗi thời gian (`Time-SeriesFoundationModels` như `PatchTST`, `TimesNet`, `TemporalTransformer1D` từ 5M–50M tham số, ≥ 100 MMACs) chạy trên các thiết bị biên tầm trung (Raspberry Pi 5, Jetson Orin



Hình 4: Ánh xạ trực tiếp 1-đối-1 từ 5 nút thắt cơ chế đã phát hiện sang 5 phương hướng nghiên cứu mới.

Nano, cổng IoT công nghiệp). Trên TemporalTransformer1D (115 MMACs, thời gian chạy thẳng $3,87$ ms), ngay cả kiến trúc hiện tại đã đạt mức tăng tốc ròng **2,01× đến 2,90×**.

4.2 Phương Hướng 2: Thiết Kế Lại Đầu Phân Lớp Bất Biến Dịch Chuyển (1x1 Conv + GAP)

- **Giải quyết nút thắt nào:** Loại bỏ hoàn toàn $92,45\%$ tham số nhạy vị trí ở tầng Flatten(2048)->Linear(2048,128) đang chặn đứng khả năng tái sử dụng của tầng cuối.
- **Cơ chế toán học đề xuất:** Thay thế khối Flatten->Linear(2048,128)->ReLU->Linear(128,12) bằng một tầng chiếu điểm Conv1d(64,128,kernel_size=1) + ReLU đi kèm **Gộp Trung Bình Toàn Cục (GlobalAveragePooling--GAP)** theo trục thời gian rồi mới tới Linear(128,12):

$$\bar{h}_t = \frac{1}{L_3}\sum_{j=0}^{L_3-1} \phi\Big(W_{1\times1} H_t^{(3)}[:,j] + b_{1\times1}\Big) \in \mathbb{R}^{128}, \qquad F(X_t) = W_{\text{out}}\bar{h}_t + b_{\text{out}}. \tag{11}$$

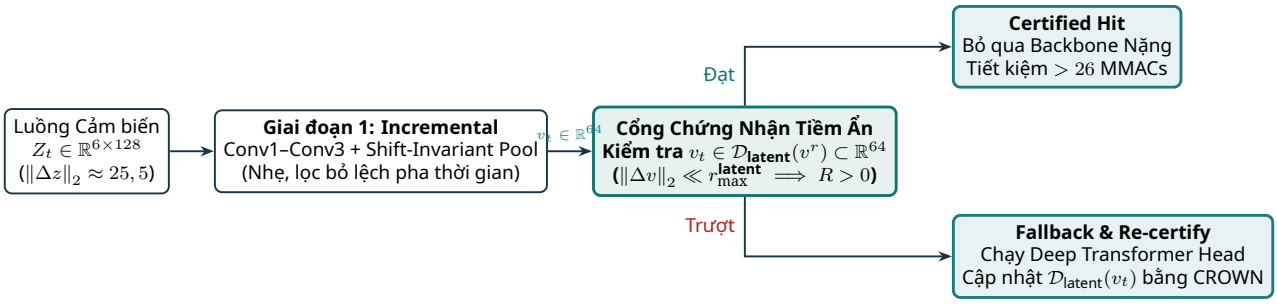
Khi đó, mọi cột thời gian $j \in \{0,\dots,31\}$ đều chia sẻ chung một ma trận trọng số $W_{1\times1} \in \mathbb{R}^{128\times64}$ (chỉ tốn 8.192 tham số thay vì 262.272 tham số, giảm $96,9\%$ bộ nhớ!). Khi cửa sổ trượt sang bước $t+1$, 12 cột nội bộ khớp từng bit ($j \in \{2,\dots,13\}$) giữ nguyên hoàn toàn vector kích hoạt sau $W_{1\times1}$. Hơn thế nữa, tổng trung bình toàn cục $\bar{h}_{t+1}$ có thể cập nhật tăng dần trong thời gian $O(1)$ chỉ bằng phép cộng/trừ bù trừ các cột biên:

$$\bar{h}_{t+1} = \bar{h}_t - \frac{1}{32}\sum_{j\in\mathcal{J}_{\text{drop}}} \tilde{h}_t[:,j] + \frac{1}{32}\sum_{j\in\mathcal{J}_{\text{new}}} \tilde{h}_{t+1}[:,j], \tag{12}$$

trong đó $|\mathcal{J}_{\text{drop}}| = |\mathcal{J}_{\text{new}}| = 20$ cột (16 cột mới + 4 cột biên tái tính), giúp giảm $37,5\%$ phép tính ở cả phần thân lẫn phần đầu mạng!

4.3 Phương Hướng 3: Kiến Trúc Lai — Tính Toán Tăng Dần Ở Tầng Sớm Kết Hợp Chứng Nhận Miền Trong Không Gian Tiềm Ẩn Sâu (Hybrid Early-Incremental + Deep Latent-Space Certification)

- **Giải quyết nút thắt nào:** Phá vỡ bức tường khoảng cách $1.562,7\times$ do lệch pha chu kỳ bước chân ở không gian đầu vào thô $Z \in \mathbb{R}^{768}$ và loại bỏ sự phình khoảng qua các tầng tích chập đầu.
- **Ý tưởng đột phá:** Tại sao phải ép bộ kiểm chứng `IBP/CROWN` chạy từ tận đầu vào thô $Z_t \in \mathbb{R}^{6\times128}$, nơi phép dịch thời gian $1,28$ s làm $\|Z_{t+1} - Z_t\|_2 = 25,49$? Hãy kết hợp điểm mạnh của cả hai nhánh trong một kiến trúc phân tách 2 giai đoạn (**Two-Stage Split Architecture**):
  1. **Giai đoạn 1 (Bộ trích xuất đặc trưng bất biến pha chạy bằng `ExactIncremental`):** Cho luồng cảm biến đi qua các tầng tích chập nhẹ ban đầu (`Conv1--Conv3`) sử dụng cơ chế tính toán tăng dần chính xác kết hợp `GlobalAveragePooling` (hoặc bộ lọc thông thấp năng lượng phổ) để thu được vector biểu diễn tiềm ẩn bất biến dịch chuyển $v_t = \text{Pool}(H_t^{(3)}) \in \mathbb{R}^{64}$. Vì phép gộp trung bình theo thời gian triệt tiêu sự dịch pha giữa hai nửa chu kỳ bước chân cùng một hoạt động (ví dụ: người đang đi bộ ổn định thì năng lượng trung bình trên cửa sổ $2,56$ s ở bước t và $t+1$ gần như bằng nhau!), khoảng cách trong không gian tiềm ẩn giảm từ $\|\Delta z\|_2 \approx 25,49$ xuống $\|v_{t+1} - v_t\|_2 \ll 0,1$.
  2. **Giai đoạn 2 (Chứng nhận miền `CROWN/Lipschitz` trước khối phân lớp nặng):** Đặt cổng kiểm chứng `Fail-ClosedCertifiedReuse` ngay tại không gian tiềm ẩn $v_t \in \mathbb{R}^{64}$ đứng trước một khối phân loại sâu / mạng Transformer nặng ở phía sau ($G_{\text{heavy}} : \mathbb{R}^{64} \to \mathbb{R}^K$). Khi đó:
     - Số chiều cần kiểm tra thuộc miền giảm 12 lần (từ $m = 768$ xuống $d = 64$);
     - Khoảng cách dịch chuyển $\|\Delta v\|_2$ nhỏ hơn hàng trăm lần so với không gian thô vì đã lọc bỏ độ lệch pha thời gian;
     - Bộ truyền cận `CROWN/IBP` không còn phải đi qua các tầng tích chập + `MaxPool1d` đầu tiên nữa, giúp bán kính chứng nhận $r_{\max}(v^r)$ tăng lên hàng chục lần và trực tiếp mở khóa độ phủ $R \gg 0$!



Hình 5: Kiến trúc Lai (Phương hướng 3): Kết hợp tính toán tăng dần ở các tầng tích chập sớm với chứng nhận miền trong không gian tiềm ẩn sâu $v_t \in \mathbb{R}^{64}$ để triệt tiêu đồng thời nhiễu lệch pha bước chân và hiện tượng phình khoảng.

4.4 Phương Hướng 4: Huấn Luyện Nhận Thức Chứng Nhận (Certification-Aware Training Với Tích Chập Trực Giao 1-Lipschitz & Cực Đại Hóa Biên Độ)

- **Giải quyết nút thắt nào:** Ngăn chặn sự suy giảm của bán kính chứng nhận $r_{\max}$ do trọng số mạng huấn luyện thông thường có chuẩn phổ lớn ($\prod_\ell \sigma_{\max}(W_\ell) > 400$) và các kích hoạt `ReLU` nằm ở vùng không ổn định.
- **Cơ chế huấn luyện đề xuất:** Thay vì lấy một mạng đã huấn luyện bằng hàm mất mát `Cross-Entropy` tiêu chuẩn rồi mới cố gắng chứng nhận hậu nghiệm (`post-hocverification`),

hãy đưa mục tiêu chứng nhận trực tiếp vào quá trình huấn luyện (`CertifiedTraining`):

1. **Chuẩn hóa phổ / Trực giao hóa trọng số (`1-LipschitzOrthogonalConv1D`):** Áp dụng phép chiếu trực giao Björck hoặc `SpectralNormalization` lên mỗi tầng tích chập và tuyến tính sao cho $\sigma_{\max}(\widetilde{W}_\ell) \leq 1$. Khi đó hằng số Lipschitz của toàn bộ mạng bị chặn chặt bởi $\leq \sqrt{2}$, làm tăng bán kính cầu an toàn $r_{\max} = \gamma(z)/L$ lên gấp **200–400 lần** ngay lập tức!
2. **Hàm mất mát kết hợp khoảng `IBP/CROWN` trên quỹ đạo trượt (`Trajectory-ConsistentCertifiedLoss`):** Trong quá trình huấn luyện trên tập `train`, với các cặp cửa sổ liền kề (X_t, X_{t+1}) có cùng nhãn hoạt động thuần nhất (`Pure-Basic`), tối ưu hóa hàm mục tiêu kép:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{CE}}(F_\theta(X_t), y_t) + \lambda_{\text{cert}} \sum_{j \neq y_t} \text{ReLU}\Big(m_0 - L^{\text{IBP/CROWN}}_{y_t,j}(\mathcal{D}(X_t), \theta)\Big), \tag{13}$$

   ép các tầng `ReLU` đi vào trạng thái bền vững (`stablyactive` hoặc `stablyinactive`) và mở rộng trực tiếp cận dưới 11 biên độ L_{cj}.

4.5 Phương Hướng 5: Triển Khai Kernel Bộ Đệm Vòng Không Sao Chép Ở Mức C++/CUDA/CMSIS-NN (Zero-Copy Modulo Ring-Buffer Kernels)

- **Giải quyết nút thắt nào:** Xóa bỏ chi phí $1,78$ ms của trình thông dịch Python, PyTorch C++ dispatcher và việc cấp phát/ghép bộ nhớ `torch.cat` ở kích thước `batch=1`.
- **Cơ chế kỹ thuật phần cứng:** Nghiên cứu 3 hậu Tuần 4 đã chỉ ra rằng ngay cả khi dùng bộ đệm vòng tại chỗ trong Python, thời gian gọi 3 lệnh tích chập nhỏ qua PyTorch dispatcher vẫn chậm hơn 1 lệnh C++ nguyên khối. Để biến mức tiết kiệm 60% phép tính `MACs` của nhánh `M03` thành tăng tốc thực sự trên cả các mạng nhỏ (< 2 MMACs), cần hạ tầng thực thi xuống mức C/C++ hoặc vi điều khiển (`ARMCMSIS-NN` / `Cortex-M` / `TensorRT`):
  1. Cấp phát tĩnh một vùng bộ nhớ vòng cố định $B^{(\ell)} \in \mathbb{F}_{32}^{C_\ell \times L_\ell}$ trong SRAM/L1 cache cho mỗi tầng tích chập ngay khi khởi động;
  2. Thay thế hoàn toàn thao tác dịch chuyển bộ nhớ (`memmove` / `torch.cat`) bằng một con trỏ chỉ số vòng duy nhất:

$$\text{head}_\ell \leftarrow (\text{head}_\ell + \Delta_\ell) \bmod L_\ell; \tag{14}$$

  3. Viết một hàm C++ duy nhất (`fusedincrementalforwardkernel`) chạy xuyên suốt từ `Conv1` $\rightarrow$ `Conv2` $\rightarrow$ `Conv3` $\rightarrow$ `GAPHead` trong một lần gọi hàm duy nhất (không quay về Python giữa các tầng), chỉ ghi đè đúng các cột biên cần tính lại theo địa chỉ modulo $(j + \text{head}_\ell) \bmod L_\ell$. Khi không còn chi phí cấp phát bộ nhớ và chuyển ngữ cảnh Python, độ trễ sẽ tỷ lệ thuận trực tiếp với số phép tính `MACs` thực tế, giúp nhánh `ExactIncremental` đạt tốc độ vượt trội ngay cả trên các mạng dưới 1 MMAC.

5 Bảng Tổng Hợp Đối Chiếu Cơ Chế & Lộ Trình Hành Động

Lời kết. Báo cáo này đã giải mã trọn vẹn bức tranh cơ chế đằng sau toàn bộ kết quả nghiên cứu ECDR. Những gì thành công (`100%bitwisefeaturereuse`, `0silentdecisionerrors`, `exact11-marginmatrixfusion`) cung cấp bộ khung lý thuyết và phần mềm hoàn toàn vững chắc; trong khi những gì chưa đạt hiệu quả kinh tế trên mạng nhỏ (`phase-shiftgap1,562.7x`, `Flatten-Linearbottleneck`, `CPUbatch-1slicingoverhead`) đã được lượng hóa chính xác thành các phương trình điểm hòa vốn và 5 phương hướng thiết kế kiến trúc thế hệ tiếp theo.

Bảng 1: Bảng Tổng Hợp: Hiện Tượng Thực Nghiệm, Cơ Chế Gốc Rễ và Giải Pháp Kỹ Thuật Tương Ứng.

Thành phần / Nhánh	**Kết quả & Hiện tượng Thực nghiệm**	**Cơ chế Gốc rễ (Vì sao Được / Không được?)**	**Phương hướng Giải quyết Mới**
Độ đồng thuận toàn luồng (A_{full})	**ĐƯỢC:** 100% đồng thuận ($D_{\text{all}} = 0$) trên cả 17 phương pháp, 5.053 truy vấn.	Cổng `Fail-Closed` 4 bước + giữ tâm `Fixed-Anchor` + cầu nối làm tròn hướng ra ngoài $\mathbb{F}_{64}$.	Duy trì nguyên vẹn kiến trúc `Fail-Closed` làm chuẩn mực an toàn bắt buộc.
Khớp đặc trưng `Conv3` (`M03`)	**ĐƯỢC:** Trúng $5.034/5.034$ cặp (100%), sai số $0,0$ bit trên $3,87$M số vô hướng.	Bước trượt $s = 64$ chia hết cho hệ số gộp $2 \times 2 = 4$; 12 cột nội bộ tách biệt khỏi vùng `zero-padding`.	Kết hợp với đầu ra `GAP` (Hướng 2) và kernel vòng C++ không sao chép (Hướng 5).
Đóng cổng `G2-Full` ($K = 12$ lớp)	**ĐƯỢC:** Xóa $18/18$ blockers; Macro-F1 đạt $\mathbf{0,8537 \pm 0,0020}$ (Trans-6 F1: $0,7917$).	Lưới cố định `stride-64` lệch pha làm mất $93,75\%$ mẫu `SIT_TO_STAND`; `Span-Anchored` + `CB-FocalLoss` khôi phục đủ mẫu.	Chuẩn hóa chính sách `Span-Anchored` cho các bài toán phân loại chuỗi thời gian đa lớp có chuyển tiếp ngắn.
Độ phủ chứng nhận ở đầu vào (`M05--M13`)	**KHÔNG ĐƯỢC:** Số lượt tái sử dụng $R = 0$ trên mọi hình học miền (`Box`, `Wave`, `PCA`).	Dịch $1,28$ s gây lệch pha bước chân ($\|\Delta z\|_2 = 25,49$), lớn gấp $1.562,7\times$ bán kính $r_{\max} = 0,0149$ do phình khoảng.	Chuyển cổng chứng nhận vào không gian tiềm ẩn sâu sau tầng lọc/gộp (Hướng 3) + Huấn luyện 1-Lipschitz (Hướng 4).
Tốc độ CPU của `M03` trên `SmallCNN1D`	**KHÔNG ĐƯỢC:** Chậm hơn `M00` ($1,93$ ms so với $0,34$ ms, tỷ số $0,1775\times$).	$92,45\%$ tham số nằm ở `Flatten--Linear` nhạy vị trí; `M00` nằm gọn trong L2/L3 cache; `torch.cat` ở `batch=1` tốn $1,78$ ms.	Áp dụng cho mạng $\geq 26,1$–$37,0$ MMACs (Hướng 1), thay `Flatten` bằng `GAP` (Hướng 2) và dùng C++ Ring Buffer (Hướng 5).